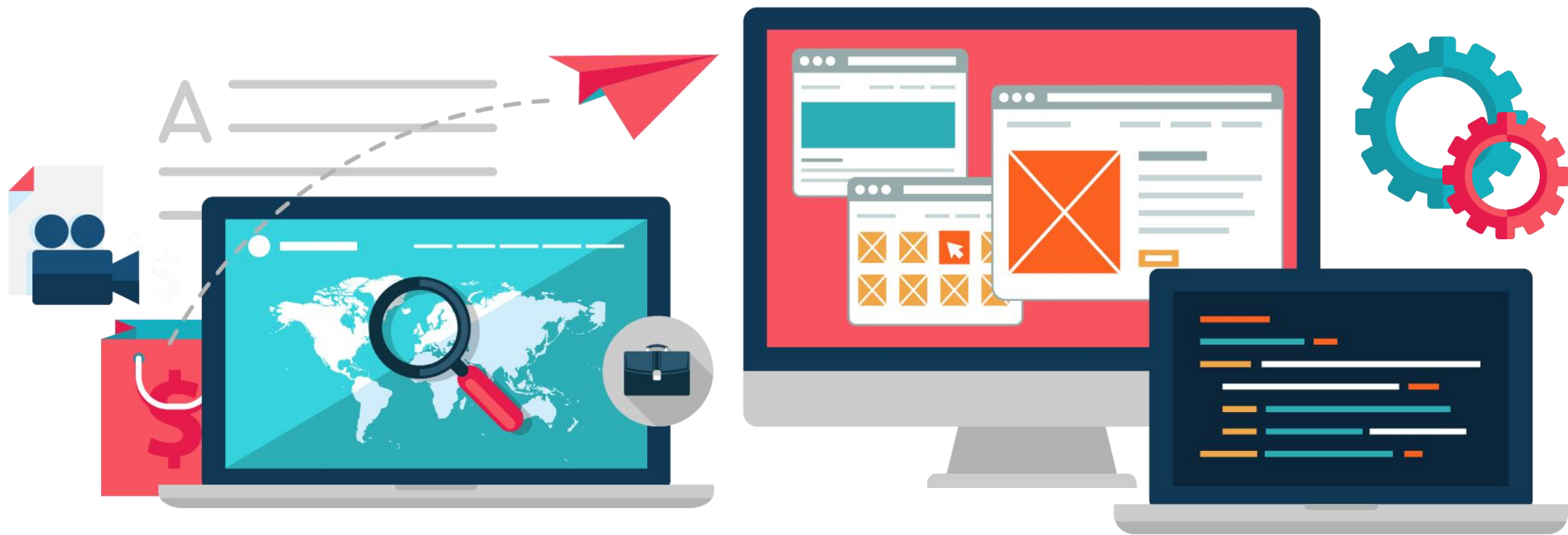


面向对象程序设计

Object Oriented Programming





构造和析构



01

自定义构造函数

02

缺省的构造函数

03

对象的初始化

04

初始化列表

05

析构函数

06

构造和析构函数的访问





- 类描述了同一类对象的共同特征，而对象是类的具体实例。同一个类的某个对象区别于另一个对象的地方就是它自身的属性，即数据成员的值(实例属性，非静态数据成员的值)
- 对象在创建的时候通常需要对非静态数据成员的初值进行设置，称为对象的初始化。同样，在对象生命期结束的时候，还可以进行一些相关的清理工作
- C++中对象的初始化和清理工作，分别由两个特殊的成员函数来完成，即构造函数和析构函数



● 构造函数

在定义对象时，**被编译系统自动调用**，用来创建对象，同时进行初始化工作。

● 格式

- 名字与类名相同
- 无返回值
- **explicit**关键字可选
- **有this**，**非static**，**非const**
- 自定义构造函数
- 默认构造函数

//例

```
class Name {  
public:  
    Name( );  
    explicit Name(int val);  
    Name(int ,int );  
private:  
    Name(Other& other);  
    ....  
};
```



- 可重载
- 可设置不同访问控制
- 可选explicit关键字
- 可带缺省参数
 - 二义性问题
- 只声明，无实现

```
class Name {  
public:  
    Name();  
    Name(int *);  
    explicit Name(int);  
    Name(Other& other);  
private:  
    Name(int val,int );  
    ....  
};  
  
void f(const Name & o) { }
```

```
int main( ) {  
    Name obj1; //Name obj1{}; 默认初始化显式调用  
    Name obj2(100); //Name obj2{100}; 直接初始化  
    Other oth;  
    Name obj3(oth); //Name obj3{oth};  
    int k = 5;  
    f(&k); // 隐式调用 //f({&k});  
    f(100); //explicit禁止隐式调用,f(Name{100});  
    f(obj2); //OK  
    Name obj4(1,2); //不可访问,改为public  
    Name obj5 = {1, 2}; //拷贝列表初始化,17保证拷贝省略  
    Name obj6{2, 3}; //直接列表初始化  
    f({10, 20}); //多参数隐式转换,生成临时Name对象  
    Name(&k); //引用k需要初始化,Name{&k};直接初始化  
    Name{Other{}}; //无匹配的构造函数,改const;直接初始化  
}
```



私有自定义构造函数 (例)

```
class Name {  
public:  
    static Name & create( ) {  
        static Name obj;  
        return obj;  
    }  
  
private:  
    Name( );  
    ....  
};  
  
int main ( )  
{  
    Name& obj =  
        Name::create( );  
    ....  
}
```

```
class Name {  
public:  
    static Name * create( ) {  
        static Name * pObj = nullptr;  
        if (pObj == nullptr) {  
            pObj = new Name;  
        }  
        return pObj;  
    }  
  
private:  
    Name( );  
    ....  
};  
  
int main ( )  
{  
    Name * pObj = Name::create( );  
    ...  
}
```



带缺省值的自定义构造函数 (例)

```
class Name {  
public:  
    Name( ) { v1=0;v2=0;}  
    Name(int v1) { n1=v1;}  
    Name(int v1,int v2=999)  
        { n1=v1;n2=v2; }  
    ....  
private:  
    int n1,n2;  
};
```



```
int main() {  
    Name obj1; //OK Name ()  
    Name obj2(200); //二义性错误(模棱两可)  
    Name obj3(100,400);  
}
```

```
class Name {  
public:  
    Name( );  
    Name(int v1);  
    Name(int v1=0,int v2=999);  
    ....  
};
```

默认构造函数：不需要提供实参就能调用的构造函数

- 无参数的，或全部参数都有默认实参的
- 用户没有声明任何构造函数时，编译器会隐式声明一个无参的、非explicit的默认构造函数
- 隐式声明的具有public、inline
- 定义延迟到第一次使用时才生成，生成发生在每个使用它的编译单元中，即按需生成

```
int main() {  
    A a; //右边任一种形式，均报错  
};
```

```
class A {  
public:  
    // inline A( ); //若无A(int)，隐式声明形式  
    A( int n ) { } //存在自定义带参构造函数  
};
```

```
class A {  
private:  
    A( int n ) { } //存在私有带参构造函数  
};
```

```
class A {  
public:  
    A( int n ); //只声明，无具体实现  
};
```

```
class A {  
public:  
    A( const A& ) { } //拷贝构造也是构造函数  
};
```




为了解决上面的问题，需要自定义默认构造函数

但是手动编写存在两个问题：

1. 程序员工作量变大；
2. 没有编译器自动生成的默认特殊成员函数效率高。

```
#include <iostream>
using namespace std;

class A
{
private:
    int a;
public:
    A() {} // 自定义默认构造函数
    A(int i)
    {
        a = i;
    }
};

int main()
{
    A obj; // 正确，默认构造函数A::A()存在
    return 0;
}
```

为了解决上述两个问题，可以使用 **=default(C++11)函数**。

在函数声明后加上 **=default**; 就可将函数声明为 **=default** 函数，编译器将为显式声明的 **=default** 函数自动生成函数体。

=default 函数既可以在类体内定义(内联)，也可以在类体外定义(非内联)，类外定义时立即生成。

该函数比自定义默认构造函数获得更高的代码效率。

=default 函数特性仅适用于类的特殊成员函数或比较运算符函数(C++20起)，并且不能有默认实参。

```
#include <iostream>
using namespace std;
```

```
class A
{
private:
    int a;
public:
    A() = default; //类内内联，延迟定义按需生成
    // X();
    A(int i)
    {
        a = i;
    }
};
A::A() = default; //类外非内联，立即生成
int main()
{
    A obj; // 正确，默认构造函数X::X()存在
    return 0;
}
```



对象的初始化(非静态数据成员初始值的设置)包括先后两个阶段:

1. 成员初始化阶段, 发生在构造函数成员初始化列表
2. 非静态数据成员赋值阶段, 发生在构造函数{ }内(通常不需要)

成员初始化阶段(无论隐式声明还是自定义的)遵循以下优先级规则:

1. 成员初始化列表显式指定
2. C++11之后允许类定义时显式指定初值(默认成员初始化器)
3. 默认初始化

// 1.成员初始化列表显式指定

```
class Card {
public:
    Card(int ald, Player& aPlayer)
        : id(ald), player(aPlayer), x(0), y(0) {}
    Card(int ald) : id(ald), player(getDefaultPlayer()),
                  x(0), y(0) {}

    // inline Card();
private:
    static Player& getDefaultPlayer(); // 返回某个全局
                                     // 或静态 Player

    int x; int y;
    const int id;
    Player& player;
};
```

初始化列表的语法: 使用 () 或 {}, 后者防止窄化转换
const成员和引用成员通常在成员初始化列表显式初始化



存在的问题：

1. 多个构造函数同一个成员初始化完全相同，代码冗余
2. 隐式声明的默认构造函数不会自动生成初始化列表，非静态数据成员都进行默认初始化
3. C++11之前，由于const成员和引用成员无法默认初始化，不能隐式声明默认构造函数

// 1.成员初始化列表显式指定

```
class Card {  
public:  
    // Card(int ald, Player& aPlayer)  
    : id(ald), player(aPlayer), x(0), y(0) {}  
    // Card(int ald) : id(ald), player(getDefaultPlayer()),  
    x(0), y(0) {}  
  
    // inline Card();  
private:  
    static Player& getDefaultPlayer(); // 返回某个全局  
    // 或静态 Player  
  
    int x; int y; // 默认初始化=未初始化，不确定值  
    const int id; // 无法默认初始化，必须显式初始化  
    Player& player; // 无法默认初始化，必须显式初始化  
};
```

成员初始化阶段(无论隐式声明还是自定义的)遵循以下优先级规则:

1. 成员初始化列表显式指定
2. C++11之后允许类定义时显式指定初值(默认成员初始化器)

✓ 本质是提供一个普遍适用的、合理的默认值

✓ 例如: 绝大多数(或所有)卡片初始位置在原点, 偏转角是0.1且不变; 绝大多数(或所有)游戏角色初始状态是存活, 血量为100

3. 默认初始化

// 2.C++11之后允许类定义时显式指定初值

```
class Card {
public:
    Card(int ald, Player& aPlayer) : id(ald), player(aPlayer) {}
private:
    float x{0}; // = 0; 或 (0) 有语法歧义不推荐
    float y{0};
    const float delta{0.1f}; //列表初始化更严格防止窄化转换
    //下面2行不合理, 存在设计缺陷而非语法错误
    const int id{100}; //不合理逻辑错误, id唯一不应指定初值
    Player& player; // 不应指定初值会导致所有对象共享资源
    static const int CARD_COUNT = 52;
    //若整型或枚举类型的静态数据成员被声明为const, 则可
    //以在类定义时指定初值, 视情况进行类外定义
    static const float AA = 99.88; // 错误, .cpp中定义
    static int bb = 10000; //错误, .cpp中定义 或:
    //static inline const float AA = 99.88f; //C++17内联定义
    //static inline int bb = 10000; //C++17内联定义
};
```



成员初始化阶段(无论隐式声明还是自定义的)遵循以下优先级规则:

1. 成员初始化列表显式指定
2. C++11之后允许类定义时显式指定初值(默认成员初始化器)

3. 默认初始化

- ✓ 成员初始化列表未显式指定且定义时未显式指定初值(无默认成员初始化器)时
- ✓ 只有非const、非引用的内置类型或具有默认构造函数的类类型成员才会使用默认初始化, 且对于局部对象的内置类型成员会产生不确定值。

// 3.默认初始化

```
class Description {
public:
    Description(int n);
    // Description( ) = default; /* 1 */
};

class Card {
public:
    Card(int ald, Player& aPlayer): id(ald), player(aPlayer) { }
    // Card(int ald, Player& aPlayer): id(ald), player(aPlayer),
    //                               desc(id) /* 2 */ { }

private:
    int x; // 默认初始化=未初始化, 不确定值或零初始化(全局/静态)
    int y; // 默认初始化=未初始化, 不确定值或零初始化(全局/静态)
    const int id; // 无法默认初始化, 必须显式初始化
    Player& player; // 无法默认初始化, 必须显式初始化
    Description desc; // 默认初始化, 调用对应的默认构造函数
    // 编译错误, Description无默认构造函数
};
```



对象的初始化(非静态数据成员初始值的设置)包括先后两个阶段:

1. 成员初始化阶段, 发生在构造函数成员初始化列表
2. 非静态数据成员赋值阶段, 发生在构造函数{ }内(通常不需要)

成员赋值阶段:

- ✓ 非静态数据成员在进入构造函数体之前就已经初始化完成
- ✓ 非初始化, 属于后续修改
- ✓ 用于特殊场景, 如需在资源分配后才能赋值(打开文件后才设置文件句柄)或对成员进行额外的逻辑检查或设置

// 构造函数{ }内成员赋值

```
class Card {  
public:  
    Card(int ald, Player& aPlayer) : id(ald), player(aPlayer)  
    {  
        x = 0; //对x赋值, 非初始化  
        y = 0; //对y赋值, 非初始化  
        id = 100; //编译错误, const不能赋值  
        player=aPlayer; //自赋值, 冗余无意义  
        Description desc; // 定义局部对象, 非对desc初始化;  
        desc = Description{ id }; // 已初始化, 重复赋初值  
    }  
private:  
    int x; int y;  
    const int id;  
    Player& player;  
    Description desc; // 默认初始化  
};
```


- ❑ 非静态数据成员初始化顺序
 - ❑ 严格按照定义顺序依次初始化
 - ❑ 先在成员初始化列表中查找是否指定了初始化方式，若找到，按指定方式初始化；
 - ❑ 若没找到，查找是否有默认成员初始化器，若找到，按默认成员初始化器初始化；
 - ❑ 若没找到，进行默认初始化，对象成员(数组)需所属类有默认构造函数；
 - ❑ 全部完成后，再进入构造函数的{ }

```
class Card {  
public:  
    Card(int ald, Player& aPlayer)  
        : x(99), id(ald), player(aPlayer), desc(ald)  
        : desc(ald), x(99), id(ald), player(aPlayer)  
        : desc(id), x(99), id(ald), player(aPlayer)  
        : y(ald), x(y), id(ald), player(aPlayer), desc(ald)  
    {  
    }  
private:  
    int x;  
    int y = 88 ;  
    const int id;  
    Player& player;  
    Description desc;  
};  
// 红色初始化列表，x不确定，y为ald，忽略类定义初值88
```

比较



非静态数据成员初始化:

- 普通数据成员: **优先**在成员初始化列表或使用默认成员初始化器初始化, 避免在构造函数体内赋值
- 常量数据成员: **必须**在成员初始化列表或使用默认成员初始化器初始化, **不能**在构造函数体内赋值
- 引用数据成员: **必须**在成员初始化列表或使用默认成员初始化器初始化, **不可**在构造函数体内赋值
- 对象数据成员: **若所属类无默认构造函数**, 则必须在初始化列表中显式初始化或使用默认成员初始化器初始化, **若所属类有默认构造函数**, 则允许默认初始化

最佳实践:

- ✓ 确保每个非静态数据成员在构造时都有明确的初始值
- ✓ **对于构造函数特有的初始化需求**, 应优先在构造函数初始化列表中显式初始化
- ✓ 对于所有构造函数共享的默认值, 可使用默认成员初始化器
- ✓ 除非特殊场景, 尽量避免在构造函数体内赋值



委托构造函数:

- ✓ 一个构造函数可以调用同一类的另一个构造函数, 避免重复代码
- ✓ 委托构造函数不能同时在初始化列表中显式初始化其他成员, 所有初始化都交由目标构造函数完成。

```
class Card {  
public:  
    // 主构造函数  
    Card(int ald, Player& aPlayer)  
        : id(ald), player(aPlayer), x(0), y(0) { }  
  
    // 委托构造函数: 仅提供 id, 使用默认的Player引用  
    Card(int ald) : Card(ald, getDefaultPlayer()) { }  
  
    // 默认构造函数委托给主构造函数  
    Card() : Card(0, getDefaultPlayer()) { }  
  
private:  
    static Player& getDefaultPlayer(); // 返回某个全局  
                                        // 或静态 Player  
  
    int x, y;  
    const int id;  
    Player& player;  
};
```

◆析构函数：负责对象销毁前，最后需要执行的清理工作(释放对象可能在它的生存期间获得的资源)，通常都是隐式调用。

◆格式：

- 名字：~ 类名
- 参数：必须为空或 void(不推荐)
- 返回值：无
- 访问控制：一般均为public
- 有this, 非static, 非const

◆当没有显式声明析构函数时，编译器会隐式声明一个inline、public的默认析构函数，与构造相同延迟生成定义，按与构造相反的顺序销毁所有非静态数据成员。

```
class Card {  
public:  
    Card(int ald, Player& aPlayer);  
    ~Card( );  
    ...  
private:  
    int x;  
    int y;  
    const int id;  
    Player& player;  
    Description desc;  
};
```

```
class A {  
public:  
    A() { pValue = new int(100); }  
    ~A( ) { delete pValue; pValue = nullptr; }  
private:  
    int *pValue;  
};
```



- ◆ 创建对象---访问构造函数

- ◆ 销毁对象---访问析构函数

- ◆ 构造函数的访问

 - ◆ 显式调用(显式创建对象)

 - ◆ 隐式调用(自动转换)

 - ◆ 对象成员的创建

- ◆ 析构函数的访问

 - ◆ 全局静态数据区、栈区：生存期结束后，自动执行(隐式调用析构函数)

 - ◆ 堆区：需显式delete(隐式调用析构函数)



对象的创建和销毁(例)

```
class B {
public:
    B( );
    explicit B(int n);
};

class A {
public:
    A( );
    A(int n);
    void do(const B & aB);
    ~A( ) { }
private:
    int num;
    B b1;
    B b2;
    static B globalB;
};
```

B A::globalB(200); //静态对象成员，存放在全局静态数据区

```
A::A( ): num(0),b2(2) { }
```

```
A::A( int n): num(n),b1(n),b2(n+1){ }
```

...

```
void func( ) {
```

```
    A a1;
```

```
    A a2(5);
```

```
    A * pa = new A(8);
```

```
    pa->do(2); //explicit禁止隐式调用，需显式调用
```

```
    delete pa;
```

```
}
```

```
int main( ) {
```

```
    A a0;
```

```
    func( );
```

```
    return 0;
```

```
}
```

该程序一共调用了多少次构造函数，析构函数呢？



本章结束